

CSC 106 - Can Computers Think? W10 - review

CSC 106: Can Computers Think?

Assignments to study

- W2-2 : for loops, input
- W2-3 : memory diagrams and tracing
- W3-3 : math facts
- W3-4 : conditionals
- W4-1 : multi-branch conditionals (Reeborg)
- W4-2 : conditionals
- W5-1 : while loops
- W5-2 : ASCII
- W6-1 : lists and strings
- W6-2 : Caesar cipher, strings
- W6-3 : lists, numbers and strings
- W7-1 : lists of numbers
- W7-2 : nested lists
- W7-3 : dictionaries and lists
- W8-1 : reading/writing files
- W8-2 : dictionaries for translation
- W8-3 : author identification

Final Exam Topics

- Values and types
- Statements and expressions
- Operators
- Function calls and definitions
- Loops: conditionals, while and for
- Strings and String operators
- Lists
- Nested Lists
- Files
- Dictionaries
- Recursion
- Sorting
- Search

Values and Types: Discussion

- How many types of values can you think of in Python?
- How would you describe a variable?
- What are the rules of variable names?
- Which of these variable names are valid? For those that are invalid... why?

```
newVar # valid
a new day # invalid - spaces
123abc # invalid - starts with numbers
for # invalid - for is a reserved word
For # valid but unusual to use cap letter at star
email@provider # invalid - at sign
_once_upon_a_time_# valid
```

Statements and Expressions: Discussion

- What is an expression?
 - a statement that provides a value
- What is a statement?

- an executable line of code
- How would you describe the difference between a statement and an expression?
 - expression evaluates to a value, returns a value, statements don't need to
- What is the correct syntax for the assignment statement?
 - `x = 10`
- How would you describe the assignment statement?
 - saving a value to a place in memory and giving it a name to access it later
- What is the value of each variable after this code runs?

```

1  var1 = "apple"
2  var2 = 2
3  var3 = "orange-"
4  var4 = var2 + 10
5  var2 = 8
6
7  print(var1)
8  print(var2)
9  print(var3)
10 print(var4)

```

```

apple
8
orange-
12

```

- Which of the following lines of code are statements?

```

x = 5    # assignment statement
x*2      # expression, still a statement
x + 10   # expression, still a statement
a = x - 3 # assignment statement
print(x) # print statement

```

Operators: Discussion

- What is an operator?
 - a built in manipulator or values, used in expressions
- Think of as many math operators as you can
 - +, -, *, **, /, //, %, in, len
 - not, and, or
 - ==, >, <, >=, <=
- What is the order of operations for math operators?
 - (), **, *//, +/-
- Which operator would make it easiest to check if a number is even or odd?
 - %
- What do these operators do with strings? + *
 - + concatenates, * replicates
- What is the value of each of the following expressions when written in Python?

3/2*4
3/(2*4)
(2*3)-4
2**3+1
2**(3+1)
2**3/4+2
"hello"*2
4//3
5%2

```
1 print(3/2*4)
2 print(3/(2*4))
3 print((2*3)-4)
4 print(2**3+1)
5 print(2**(3+1))
6 print(2**3/4+2)
7 print("hello"*2)
8 print(4//3)
9 print(5%2)
```

```
6.0
0.375
2
9
16
4.0
hellohello
1
1
```

Function Basics: Discussion

- In your own words, how would you describe a function?
 - a collection of statements saved to a name to be reused to accomplish a specific task
- What is the syntax for calling a function?
 - `<function_name>()`
- What is the syntax for defining a function?
 - `def <function_name>():`
 - https://reeborg.ca/reeborg.html?lang=en&mode=python&menu=worlds%2Fmenus%2Freeborg_intro_en.json&name=Alone&url=worlds%2Ftutorial_en%2Falone.json#
- Try to come up with the names of at least three built-in functions from Reeborg's World
 - `move()`, `take()`, `put()`, `turn_left()`, `think(100)`
- Try to come up with the names of at least three built-in functions from Python's Turtle module
 - `turtle.pu()`, `turtle.forward()`, `turtle.right()`
 - <https://docs.python.org/3/library/turtle.html>
- Try to come up with the names of at least three built-in functions from Basic Python (i.e. without Turtle or any other modules or libraries)
 - `print()`, `max()`, `len()`

Functions with parameters: Discussion

- In your own words, how would you describe the difference between reserved words and built-in functions in Python?
 - reserved words / keywords are words python interprets a specific way so cannot be used otherwise
 - built-in functions are function definitions, lines of code, saved during python installation
- What is the syntax to call a function that takes parameters?
 - `<function_name>(<argument>)`
- What is the syntax to define a function that takes parameters?
 - `def <function_name>(<param_name>):`
- Try to name at least three built-in Python functions that take parameters
 - `range()`, `input()`, `max()`
- The `int()` function can be used to convert floating point numbers to integers through truncation. What is the difference between rounding and truncation?
 - rounding can go up or down depending on the value - nearest whole number
 - truncating ignores the value of decimal places, always rounds down
 - Can the `int()` function be used to convert a string that represents a floating point number to an integer? # Nope
- Does the program below work? Why or why not?
 - yes, multiplies two numbers, both integers

```
1 def display_product(a, b):
2     product = a*b
3     print(product)
4
5 display_product(5, 10)
```

50

- Does the program below work? Why or why not?
 - yes, multiplies two numbers and `*` can handle ints and floats, return float

```

1 def display_product(a, b):
2     product = a*b
3     print(product)
4
5 display_product(13, 0.5)

```

6.5

- Does the program below work? Why or why not?
 - not really, replicates string rather than calculating product

```

1 def display_product(a, b):
2     product = a*b
3     print(product)
4
5 display_product(2, "4")

```

44

- Does the program below work? Why or why not?
 - assume that the user actually enters integer values
 - yes, works because strings from `input()` are converted to integers with `int()`

```

1 def display_product(a, b):
2     product = a*b
3     print(product)
4 x = 2 #int(input("What's the first factor?"))
5 y = 4 #int(input("What's the second factor?"))
6 display_product(x, y)

```

8

Functions with return values: Discussion

- In your own words, describe the difference between a void function and a fruitful function.
 - void functions don't produce a value, return `None`
 - fruitful functions return values

- What is the syntax to store the value returned by a fruitful function?

- `x = foo()`

- What is the syntax to define a function that returns a value?

```
def foo():  
    return 10
```

- What happens if you assign a variable to the result of a void function?
 - variable holds value `None`
- What will the values of `int_1`, `int_2`, and `int_3` be after the code below runs? Can you explain why?

```
1 def func1(a, b):  
2     a = a*b  
3     return a  
4 int_1 = 2  
5 int_2 = 3  
6 int_3 = func1(int_1, int_2)  
7  
8 print(int_1)  
9 print(int_2)  
10 print(int_3)
```

2
3
6

Conditionals: Discussion

- What is the value of `x` after this code runs?

```
1 x = 5  
2 if x > 2:  
3     x = -3  
4     x = 1  
5 else:  
6     x = 3  
7     x = 2
```



```
8  
9 print(x)
```

1

- What is the value of x after this code runs?

```
1 x = 1  
2 if x > 2:  
3     x = -3  
4     x = 1  
5 else:  
6     x = 3  
7     x = 2  
8  
9 print(x)
```

2

- Given the function definition for is_odd below, what will the result of the function calls following it be?

```
1 def is_odd(x):  
2     if x % 2 == 1:  
3         return True  
4     else:  
5         return False  
6  
7 print(is_odd(1))  
8 print(is_odd(2))  
9 print(is_odd(3.5))  
10 #is_odd("abc")
```

True
False
False

- Consider the following function definition for is_odd. Can you rewrite the function so that it only uses one line after "def is_odd(x):"?

```

1 def is_odd(x):
2     if x % 2 == 1:
3         return True
4     else:
5         return False
6
7 print(is_odd(2))

```

False

```

1 def is_odd(x):
2     return (x % 2) == 1
3 print(is_odd(2))

```

False

- What gets printed when the following code runs?

```

1 grade = 100
2 if (grade >= 90):
3     print("You got an A!")
4 if (grade >= 80):
5     print("You got a B!")
6 else:
7     print("You got another grade.")

```

You got an A!

You got a B!

- What gets printed when the following code runs?

```

1 grade = 100
2 if (grade >= 90):
3     print("You got an A!")
4 elif (grade >= 80):
5     print("You got a B!")
6 else:
7     print("You got another grade.")

```

You got an A!

- How can you fill in the blanks so the following payscale is followed?

	experience
age	0
<18	\$15.25
18+	\$15.25

```
1 experience = 0
2 age = 0
3
4 if experience == 0:
5     wage = 15.25
6 elif (_1_):
7     if (_2_):
8         wage = 17.50
9     else:
10        wage = 16.25
11 else:
12    wage = 16.0
```

```
1 experience = 4
2 age = 17
3
4 if experience == 0:
5     wage = 15.25
6 elif (age >= 18 ):
7     if (experience > 2):
8         wage = 17.50
9     else:
10        wage = 16.25
11 else:
12    wage = 16.0
13
14 print(wage)
```

16.0

- What will the value of y be after this code runs?

```

1 x = 8
2 y = (x > 4) and ((not(x>8))or(x==5))
3 print(y)

```

True

While Loops: Discussion

- Under what conditions will the loop keep running? Are there any problems with the loop?
 - will run forever or until input cannot be converted into a float
 - all numbers are either greater than 10 or less than 20 because $10 < 20$

```

1 x = float(input("Enter a number between 10 and 20: "))
2 sum = 0
3
4 while x > 10 or x < 20:
5     sum += x
6     x = float(input("Enter another number between 10 and 20: "))
7
8 print("The sum of all the numbers you entered is", sum)

```

- Under what conditions will the loop keep running? Are there any problems with the loop?
 - will not enter the loop because x is initialized less than 10

```

1 x = 0
2 sum = 0
3
4 while x > 10 and x < 20:
5     x = float(input("Enter a number between 10 and 20: "))
6     sum += x
7
8 print("The sum of all the numbers you entered is", sum)

```

- Under what conditions will the loop keep running? Are there any problems with the loop?

- will never enter the loop because no number can be less than 0 and greater than 10
- will collect one piece of input but won't add it to sum

```

1 x = float(input("Enter a number less than 0 or bigger than 10: "))
2 sum = 0
3
4 while x > 10 and x < 0:
5     x = float(input("Enter a number less than 0 or bigger than 10: "))
6     sum += x
7
8 print("The sum of all the numbers you entered is", sum)

```

- How would you describe the purpose and function of the following loop?
 - ensure the answer is not an Imaginary number, is in the set of Real numbers

```

1 import math
2
3 x = float(input("Find the square root of which number? "))
4
5 while x < 0:
6     x = float(input("Find the square root of which number? "))
7
8 print(math.sqrt(x))

```

- Fill in the blanks to make the program function.

```

1 print("Computing the sum of positive numbers.")
2
3 x = float(input("Enter the first number (-1 to stop)"))
4 sum = 0
5
6 while __1__:
7     x = float(input("Enter the next number (-1 to stop)"))
8     __2__
9
10 print(sum)

```

```

1 print("Computing the sum of positive numbers.")
2
3 x = float(input("Enter the first number (-1 to stop)"))
4 sum = x
5
6 while not x == -1:
7     x = float(input("Enter the next number (-1 to stop)"))
8     if x > 0:
9         sum += x
10
11 print(sum)

```

basic string operations

- + : concatenation
- * : duplication/multiplication
- in : checking containment
- len : getting length
- <string name>[<i>] : indexing
- <string name>[<start>:<stop>] : slicing

string operations

- ord(<char>) : get the ASCII value of a character
- chr(<integer>): get the character associated with the integer
- "string".format(<args>) : format the string
- "a string".split('<arg>') : split on "
- " string ".strip(<args>) : remove at the ends
- "STring".lower() : convert all letters to lowercase
- "STring".upper() : convert all letters to uppercase
- "string".isalpha() : checks if all characters are letters

Strings: Discussion

- What will the value of mystring be after the following code runs?

```

1 mystring = ""
2
3 for i in range(1, 6):
4     mystring += str(i) + " "
5
6 print(mystring)

```

1 2 3 4 5

- What will the value of mylist be after the following code runs?

```

1 mystring = "They're celebrating the triumphant return of their heroes."
2
3 mylist = mystring.split()
4 print(mylist)

```

["They're", 'celebrating', 'the', 'triumphant', 'return', 'of', 'their', 'heroes.']

- What will the value of mylist be after the following code runs?

```

1 mystring = "It is a period of civil war. Rebel spaceships, striking from a
  ↪ hidden base, have won their first victory against the evil Galactic
  ↪ Empire"
2
3 mylist = mystring.split(".")
4 print(mylist)

```

['It is a period of civil war', ' Rebel spaceships, striking from a hidden base, have won the', 'Empire']

- What will the value of mystring be after the following code runs?

```

1 mystring = "   abc 123   "
2
3 mystring.strip()
4 print(mystring)

```

abc 123

- What will the value of mystring be after the following code runs?

```

1 mystring = "   abc 123   "
2
3 mystring = mystring.strip()
4 print(mystring)

```

abc 123

- What will the value of mystring be after the following code runs?

```

1 mystring = "3.14159265359 starts pi, while e starts with 2.7182818284"
2
3 mystring = mystring.strip("123456789. ")
4 print(mystring)

```

starts pi, while e starts with

- What will the value of new_string be after the following code runs?

– z transforms to { instead of a

```

1 mystring = "mdudq fnmmz fhud xnt to, mdudq fnmmz kds xnt cnvm, mdudq fnmmz
  ↪ qtm zqntmc zmc cdrdqs xnt"
2
3 new_string = ""
4 for char in mystring:
5     if char.isalpha():
6         new_string += chr(ord(char) + 1)
7     else:
8         new_string += char
9 print(new_string)

```

never gonn{ give you up, never gonn{ let you down, never gonn{ run {round {nd desert you

```

1 mystring = "mdudq fnmmz fhud xnt to, mdudq fnmmz kds xnt cnvm, mdudq fnmmz
  ↪ qtm zqntmc zmc cdrdqs xnt"
2
3 new_string = ""
4 for char in mystring:
5     if char.isalpha():
6         new_string += chr(((ord(char)-96) % 26) + 97)

```



```

7     else:
8         new_string += char
9     print(new_string)

```

never gonna give you up, never gonna let you down, never gonna run around and desert you

list operations

- + : concatenation
- * : duplication/multiplication
- in : checking containment
- len : getting length
- [] : indexing
- append() : appending
- extend() : extending
- <list>[start:stop] : slicing

Lists: Discussion

- What will the value of mylist be after this code snippet has been executed?

```

1 mylist = ["a string", 123, False, 4.5]
2 mylist[2] = True
3
4 print(mylist)

```

['a string', 123, True, 4.5]

- What will the code below print to the screen?

```

1 numlist = [1, 231, 85, 4]
2 for num in numlist:
3     print(num, end= "")

```

1231854

- What will the value of nums2 be after this code snippet runs?

```
1 nums1 = [1, 3, 5, 7, 9, 11, 13]
2 nums2 = nums1[2:5]
3 print(nums2)
```

[5, 7, 9]

- What will the value of nums3 be after this code snippet runs?

```
1 nums1 = [1, 3, 5]
2 nums2 = [2, 8, 10]
3 nums3 = nums1 + nums2
4 print(nums3)
```

[1, 3, 5, 2, 8, 10]

- What will the code below print to the screen?

```
1 mylist = [1] * 4
2 print(mylist)
```

[1, 1, 1, 1]

- What will the value of nums be after this code snippet runs?

```
1 nums = [1, 3, 5]
2 nums.append(8)
3 print(nums)
```

[1, 3, 5, 8]

- What will the value of nums be after this code snippet runs?

```
1 nums = [1, 3, 5]
2 nums.append([8, 9])
3 print(nums)
```

[1, 3, 5, [8, 9]]

- What will the value of nums be after this code snippet runs?

```

1 nums = [1, 3, 5]
2 nums.extend([8, 9])
3 print(nums)

```

[1, 3, 5, 8, 9]

- Will the following code run? If not, why?

```

1 mylist = [7, 8, 9, "123", "run"]
2 mylist.sort()
3 print(mylist)

```

- Write a snippet of code to compute the average of the numbers contained in the list nums.

```

1 nums = [991, 563, 647, 446, 929]
2
3 sum = 0
4 for n in nums:
5     sum += n
6 avg = sum / len(nums)
7 print(avg)

```

715.2

- What will the value of names be after this code runs?

```

1 def sort_and_pop(list_in):
2     list_in.sort()
3     list_in.pop()
4
5 names= ["Zeke", "eren", "Sasha", "mikasa"]
6 sort_and_pop(names)
7 print(names)

```

['Sasha', 'Zeke', 'eren']

- What will the following code print?

```

1 def list_change(in_list):
2     in_list = [9, 10, 11]
3     print(in_list)
4
5 mylist = ["W", "A", "H"]
6 print(mylist)
7 list_change(mylist)
8 print(mylist)

```

```

['W', 'A', 'H']
[9, 10, 11]
['W', 'A', 'H']

```

- What will the value of mylist be after the following code runs?

```

1 def list_change(in_list):
2     print(in_list)
3     in_list = in_list + [4, 5, 6]
4
5 mylist = ["e", "f", "g"]
6 list_change(mylist)
7 print(mylist)

```

```

['e', 'f', 'g']
['e', 'f', 'g']

```

- What will the value of mylist be after the following code runs?

```

1 def list_change(in_list):
2     print(in_list)
3     in_list.extend(["a", "b", "c"])
4
5 mylist = [1, 2, 3]
6 list_change(mylist)
7 print(mylist)

```

```

[1, 2, 3]
[1, 2, 3, 'a', 'b', 'c']

```

- What will the value of mylist be after the following code runs?

```

1 def list_change(in_list):
2     in_list.append("e")
3     in_list.append("f")
4     in_list.append("g")
5     print(in_list)
6
7 mylist = [4, 5, 6]
8 list_change(mylist)
9 print(mylist)

```

[4, 5, 6, 'e', 'f', 'g']
 [4, 5, 6, 'e', 'f', 'g']

- What will the value of mylist be after the following code runs?

```

1 def list_change(in_list, first_char):
2     start = ord(first_char)
3     for i in range(len(in_list)):
4         in_list[i] = chr(start + i)*(i + 1)
5
6 mylist = [1, 2, 3]
7 list_change(mylist, "b")
8 print(mylist)

```

['b', 'cc', 'ddd']

- What will the value of mylist be after the following code runs?

```

1 def list_change(in_list):
2     in_list = in_list[1:4]
3
4 mylist = ["a", "b", "c", "d", "e", "f", "g"]
5 list_change(mylist)
6 print(mylist)

```

['a', 'b', 'c', 'd', 'e', 'f', 'g']

nested lists: discussion

Write a loop or set of nested loops to print the following nested list in a grid.

```
1 mylist = [[1, 3, 5],
2           [2, 4, 6, 8, 10],
3           [3, 5, 7, 11],
4           [2, 4, 8, 16, 32, 64]]
5
6
7 for row in mylist:
8     for item in row:
9         print(item, end = '\t')
10    print('\n')
```

```
1   3   5
2   4   6   8  10
3   5   7  11
2   4   8  16 32 64
```

- What will the value of mylist be after the following code runs?

```
1 mylist = []
2
3 for i in range(5):
4     mylist.append([])
5     for j in range(3):
6         mylist[i].append((i+1)**j)
7
8 print(mylist)
```

```
[[1, 1, 1], [1, 2, 4], [1, 3, 9], [1, 4, 16], [1, 5, 25]]
```

- What will the following code print?

```

1  mylist = [[1, 3, 5],
2  [2, 4, 6, 8, 10],
3  [3, 5, 7, 11],
4  [2, 4, 8, 16, 32, 64]]
5
6  for line in mylist:
7      if 3 in line:
8          print(line)

```

[1, 3, 5]
[3, 5, 7, 11]

- What will the following code print?

```

1  mylist = ["a", "b", "c",
2           "d", "e", "f",
3           "abc", "def", "ghi",
4           "ball", "a", "hall"],
5           "abcdefg"]
6
7  for line in mylist:
8      if 'a' in line:
9          print(line)

```

['a', 'b', 'c']
['ball', 'a', 'hall']
abcdefg

- What will the value of mylist be after the following code runs?

```

1  list1 = []
2  list2 = []
3  mylist = []
4
5  for num_1 in list1:
6      line = []
7      for num_2 in list2:
8          line.append(num_1*num_2)
9      mylist.append(line)
10
11 print(mylist)

```

[]

- What will the following code print?

```
1 mylist = [["night", "fear", "love", "here", "flier"],
2           ["gift", "gong", "hand", "bound", "playa"],
3           ["greetings", "fire", "wyvern", "tried"],
4           ["yacht", "room", "dour"],
5           ["unified", "opinions"]]
6
7 for i in mylist:
8     for j, word in enumerate(i):
9         print(word[j], end="")
10    print(end=" ")
```

never gonna give you up

- Are there any problem with the following code? If so, explain.
 - runs, just created nested list 3 deep
 - last append could be changed to extend to make 2D

```
1 mylist = []
2
3
4 for i in range(1, 11):
5     list1 = []
6     for j in range(1, 11):
7         list2 = []
8         for k in range(1, 11):
9             list2.append(i*j*k)
10            list1.append(list2)
11    mylist.append(list1)
12
13 print(mylist)
```

[[[1, 2, 3, 4, 5, 6, 7, 8, 9, 10], [2, 4, 6, 8, 10, 12, 14, 16, 18, 20], [3, 6, 9, 12, 15, 18, 21, 24, 27, 30], [4, 8, 12, 16, 20, 24, 28, 32, 36, 40], [5, 10, 15, 20, 25, 30, 35, 40, 45, 50], [6, 12, 18, 24, 30, 36, 42, 48, 54, 60], [7, 14, 21, 28, 35, 42, 49, 56, 63, 70], [8, 16, 24, 32, 40, 48, 56, 64, 72, 80], [9, 18, 27, 36, 45, 54, 63, 72, 81, 90], [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]], [2, 4, 6, 8, 10, 12, 14, 16, 18, 20], [3, 6, 9, 12, 15, 18, 21, 24, 27, 30], [4, 8, 12, 16, 20, 24, 28, 32, 36, 40], [5, 10, 15, 20, 25, 30, 35, 40, 45, 50], [6, 12, 18, 24, 30, 36, 42, 48, 54, 60], [7, 14, 21, 28, 35, 42, 49, 56, 63, 70], [8, 16, 24, 32, 40, 48, 56, 64, 72, 80], [9, 18, 27, 36, 45, 54, 63, 72, 81, 90], [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]]]

- There is a problem with the behavior of this code. Please explain the problem.
 - includes -1 as a record sale
 - missed the last sale recorded for each day


```

1 def record_sales(in_list):
2     sale = float(input("Enter the next sale (-1 to stop): "))
3     while not (sale < 0):
4         sale = float(input("Enter the next sale (-1 to stop): "))
5         in_list.append(sale)
6
7 sales_records = []
8 enter_record = input("Enter records for a new day (y/n)? ")
9 while enter_record.lower()[0] == "y":
10     sales_records.append([])
11     record_sales(sales_records[len(sales_records)-1])
12     enter_record = input("Enter records for a new day (y/n)? ")
13
14 print(sales_records)

```

- Fill in the blanks, so this asks for new entries until "n" is entered

```

1 def add_record():
2     fname = input("First name: ")
3     lname = input("Last name: ")
4     phone = input("Phone number: ")
5     return [fname, lname, phone]
6
7 customer_list = []
8 enter_record = input("Enter new record (y/n)? ")
9 while enter_record.lower()[0] != "n":
10     customer_list.append(.....)
11     .....
12
13 print(customer_list)

```

```

1 def add_record():
2     fname = input("First name: ")
3     lname = input("Last name: ")
4     phone = input("Phone number: ")
5     return [fname, lname, phone]
6
7 customer_list = []
8 enter_record = input("Enter new record (y/n)? ")
9 while enter_record.lower()[0] != "n":
10     customer_list.append(add_record())
11     enter_record = input("Enter new record (y/n)? ")

```

```
12  
13 print(customer_list)
```

Basic File Operations

- Getting access
 - `open("<filename>",mode="a/r/w")` : get access to the file
- Reading contents
 - `read()` : return the remaining contents as a string
 - `readline()` : return the next line's contents as a string
 - `readlines()` : return the remaining contents as a list of strings
- Writing new stuff
 - `write(<a string>)` : write a single string to the file
 - `print(<args>,file=<f>)` : write information to the file with print formatting
- Closing it: the most important one... do not forget
 - `close()` : close the file and save its contents to long-term storage

Files: Discussion

test_file.txt

Three Rings for the Elven-kings under the sky,
Seven for the Dwarf-lords in their halls of stone,
Nine for Mortal Men doomed to die,
One for the Dark Lord on his dark throne
In the Land of Mordor where the Shadows lie.
One Ring to rule them all, One Ring to find them,
One Ring to bring them all, and in the darkness bind them,
In the Land of Mordor where the Shadows lie.

- What are the three modes which can be used when opening a file?
 - 'r' read, 'w' write, 'a' all/ read and write
- What type of data does contents hold after this code runs?

- list

```
1 infile = open("test_file.txt", "r")
2 contents = infile.readlines()
3 infile.close()
4 print(type(contents))
```

<class 'list'>

- What type of data does contents hold after this code runs?

- string

```
"""{python{ infile = open("test_file.txt", "r") contents = infile.readline() infile.close()
print(type(contents)) print(contents)
```

* What type of data does contents hold after this code runs?

+ string

```
::: {.cell execution_count=60}
``` {.python .cell-code}
infile = open("test_file.txt", "r")
contents = infile.read()
infile.close()
```

```
print(type(contents))
print(contents)
```

<class 'str'>

```
Three Rings for the Elven-kings under the sky,
Seven for the Dwarf-lords in their halls of stone,
Nine for Mortal Men doomed to die,
One for the Dark Lord on his dark throne
In the Land of Mordor where the Shadows lie.
One Ring to rule them all, One Ring to find them,
One Ring to bring them all, and in the darkness bind them,
In the Land of Mordor where the Shadows lie.
```

:::

- test\_file.txt

Three Rings for the Elven-kings under the sky,  
Seven for the Dwarf-lords in their halls of stone,  
Nine for Mortal Men doomed to die,  
One for the Dark Lord on his dark throne  
In the Land of Mordor where the Shadows lie.  
One Ring to rule them all, One Ring to find them,  
One Ring to bring them all, and in the darkness bind them,  
In the Land of Mordor where the Shadows lie.

- What will the following code display?

```
1 infile = open("test_file.txt", "r")
2 lines = infile.readlines()
3 infile.close()
4
5 for line in lines:
6 print(line.strip())
```

Three Rings for the Elven-kings under the sky,  
Seven for the Dwarf-lords in their halls of stone,  
Nine for Mortal Men doomed to die,  
One for the Dark Lord on his dark throne  
In the Land of Mordor where the Shadows lie.  
One Ring to rule them all, One Ring to find them,  
One Ring to bring them all, and in the darkness bind them,  
In the Land of Mordor where the Shadows lie.

- How many strings will be contained in lines after this code runs?

```
1 infile = open("test_file.txt", "r")
2 lines = infile.readlines()
3 infile.close()
4
5 for line in lines:
6 print(line.strip())
7
8 print(len(lines))
```

Three Rings for the Elven-kings under the sky,  
Seven for the Dwarf-lords in their halls of stone,  
Nine for Mortal Men doomed to die,  
One for the Dark Lord on his dark throne  
In the Land of Mordor where the Shadows lie.  
One Ring to rule them all, One Ring to find them,  
One Ring to bring them all, and in the darkness bind them,  
In the Land of Mordor where the Shadows lie.  
8

## file2.txt

This was a triumph.  
I'm making a note here:  
huge success.  
It's hard to overstate  
My satisfaction.  
Aperture Science.  
We do what we must  
Because we can.  
For the good of all of us.  
Except the ones who are dead.

- What will the following code print?

```
1 infile = open("file2.txt", "r")
2 contents = infile.read()
3 infile.close()
4 contents = contents.split(".")
5
6 for item in contents:
7 print(item.strip())
```

This was a triumph  
I'm making a note here:  
huge success  
It's hard to overstate  
My satisfaction  
Aperture Science  
We do what we must  
Because we can

For the good of all of us  
Except the ones who are dead

- What will the file outfile contain after this code runs?

– 0 0                      1 1                      2 4                      3 9

```
1 outfile = open("out.txt", "w")
2
3 for i in range(4):
4 print(i, i**2, file=outfile)
5 outfile.close()
```

## Basic Disctionary Operations

- Consider the following dictionary: `d = {'z': 430, 116: 'e'}`
- `d.keys()` : get all of the dictionary's keys
- `d.values()` : get all of the dictionary's values
- `d.items()` : get all of the dictionary's key:value pairs

## dictionary: discussion

- What will the following code print?

```
1 d = {'briar':128, 'logan':45, 'skylar':77, 'jady':144}
2
3 print(d["logan"])
```

45

- What of the following is best implemented as a dictionary instead of a list?
  1. The amount of time elapsed between presses of a button
    - list
  2. The names on a class roster
    - list (could be dups and just one data type, maybe dict if first and last names)

3. The names of trees and descriptions of their leaves

- dict

4. 42 randomly generated floating point numbers

- list

- What will the following code print?

```
1 d = {'bamboo':"bambú", 'mirror':"espejo", 'day':"día", 'book':"libro"}
2 print(d["bamboo"])
```

bambú

- Will the following code run? Why or why not?
  - will run but dont add entries to the dict unless entered with ; not :

```
1 d = {}
2
3 usr_in = input("Enter a key:value pair")
4 usr_in = usr_in.split(";")
5 while len(usr_in) == 2:
6 d[usr_in[0]] = usr_in[1]
7 usr_in = input("Enter a key:value pair")
8 usr_in = usr_in.split(";")
9
10 print(d)
```

- Assume that d was defined as a dictionary previously. How would you describe what this code does?
  - keeps a running dictionary of word definitions

```
1 d = {}
2 usr_in = input("Enter the word you want to look up")
3 while len(usr_in) > 0:
4 if usr_in in d.keys():
5 print(d[usr_in])
6 else:
7 print("word unknown")
8 d[usr_in] = input("Please provide a definition")
9 usr_in = input("Enter the word you want to look up")
10
11 print(d)
```

- Given the snippet of code below, write another snippet of code to add the key "pine" to the dictionary with the value 45.

```
1 trees = {'oak': 3, 'walnut': 97, 'maple': 24, 'birch': 1}
```

```
1 trees["pine"] = 45
```

- What does the following code print?

```
1 names = {'Janice': 5, 'Emily': 3, 'John': 7, 'Eleanor': 2}
2 list_o_names = []
3 for name in names:
4 if names[name] > 5:
5 list_o_names.append(name)
6 print(list_o_names)
```

['John']

- This works:
  - d[("a", 123, False)] = True
- But this does not:
  - d[("a", [1,2,3], False)] = True
- Why?
  - lists can't be dictionary keys
- Assume that d was defined as a dictionary previously. How do you think this dictionary is intended to be used?
  - keep track of movies and directors

```
1 d = {}
2 in_str = "George Lucas: Star Wars- Episode IV - A New Hope"
3 in_str = in_str.split(":")
4
5 d[in_str[0]].append(in_str[1])
```

- Is the following code valid? Why or why not? How do you think this dictionary is intended to be used?
  - valid, creates nested dictionary



- keeps track of lines of a quote

```
1 d = {}
2 d["lotr"] = {"l1" : "Three Rings for the Elven-kings under the sky",
3 "l2" : "Seven for the Dwarf-lords in their halls of stone",
4 "l3" : "Nine for Mortal Men doomed to die",
5 "l4" : "One for the Dark Lord on his dark throne"}
6
7 print(d)
```

```
{'lotr': {'l1': 'Three Rings for the Elven-kings under the sky', 'l2': 'Seven for the Dwarf-
lords in their halls of stone', 'l3': 'Nine for Mortal Men doomed to die', 'l4': 'One for th
```

- Is the following code valid? Why or why not?
  - What do you think this is being used for?
  - yes, valid
  - keeps track of links to images

```
1 d = {}
2 d["img1"] = "https://i.imgur.com/cC2RzZk.jpg"
3 d["img2"] = "https://i.imgur.com/AlIze1P.jpg"
4 d["img3"] = "https://i.imgur.com/SJActxD.jpg"
5 d["img4"] = "https://i.imgur.com/a2dXlxd.jpg"
6
7 print(d)
```

```
{'img1': 'https://i.imgur.com/cC2RzZk.jpg', 'img2': 'https://i.imgur.com/AlIze1P.jpg', 'img3
```

- Is the following code valid? Why or why not?
  - yes valid, new items can be added to dictionaries even if key doesn't previously exist

```
1 d = {}
2 d[0] = 123
3 d[1] = "abc"
4 d[2] = "xyz"
5 d[3] = False
6
7 print(d)
```

```
{0: 123, 1: 'abc', 2: 'xyz', 3: False}
```

- Is the following code valid? Why or why not?
  - Assume both d and e were previously defined as dictionaries.
  - yes valid, iterates through keys of d and adds to e

```
1 d = {}
2 d[0] = 123
3 d[1] = "abc"
4 d[2] = "xyz"
5 d[3] = False
6 e={}
7 for key in d.keys():
8 e[key] = d[key]*2
9
10 print(d)
11 print(e)
```

```
{0: 123, 1: 'abc', 2: 'xyz', 3: False}
{0: 246, 1: 'abccabc', 2: 'xyzxyz', 3: 0}
```

## Which of the following is a difference between lists and dictionaries?

1. List elements cannot be mutable, but dictionary values can be.
  - False, both are mutable
2. Assigning to an index that does not exist in a list is an error, but assigning a value to a key that doesn't exist in a dictionary is not
  - True
3. A list can contain a dictionary as one of its elements, but a dictionary cannot contain a list as one of its value
  - False, lists can contain dicts and dicts can contain lists as values (not keys)
4. There is a dict constructor that creates a dictionary from a suitable object, but there is no list constructor that does the same.
  - False, there are constructors for both

## Recursion: Discussion

- In your own words, describe recursion as it pertains to computer programming.
  - when a function calls itself
  - used when problems can be solved by solving simpler versions of themselves first
- Can you think of any examples in real life where you see recursive structures?
  - shells, leaves, pictures of pictures
- In your own words describe halting as it pertains to computer programming.
  - when the state change reaches the base case and returns
- Will the following recursive function always halt? Why or why not?
  - it will halt if num is an integer since num will eventually be multiple of 13
  - will not halt if num is a float as modulo operator will never produce 0

```
1 def func(num, adjust=0):
2 if num % 13 ==0:
3 return adjust
4 else:
5 return func(num+1, adjust+1)
```

- Will the following recursive function always halt? Why or why not?
  - no because base case is when num is less than 1, but state change increases num

```
1 def func(num):
2 if num <= 1:
3 return num
4 else:
5 return num + func(num+1)
```

- Below is an example of a recursive function to calculate Fibonacci numbers.
  - What do you think the pros and cons of this method are?
  - only takes one argument
  - adding two previous values is intuitive to definition of Fibonacci series

```

1 def fib(num):
2 if num <= 1:
3 return num
4 else:
5 return fib(num-1) + fib(num-2)
6
7 fib(6)

```

8

- Below is another example of a recursive function to calculate Fibonacci numbers.
  - Is this method better or worse than the last one? Why?
  - only one recursive call, easier to trace
  - requires extra arguments, but provides default values so user does not need to input them

```

1 def fib(num, val=1, prev=0):
2 if num == 0:
3 return prev
4 return fib(num-1, val+prev, val)
5
6 fib(6)

```

8

## Remember at least one of the sort methods we discussed

- + use the online simulators to be able to describe the process of one of these
- + you won't have to provide code syntax for implementations, but describe the process in english
- + just need to know 1 sort and 1 search, you can choose which

- Bubble Sort
- Merge Sort
- Selection Sort
- Insertion Sort
- Quick Sort

- Bucket / Bin Sort
- Linear Search
- Binary Search

## **Final Exam Topics**

- Values and types
- Statements and expressions
- Operators
- Function calls and definitions
- Loops: conditionals, while and for
- Strings and String operators
- Lists
- Nested Lists
- Files
- Dictionaries
- Recursion
- Sorting
- Search